

CS 486/586

Chapter 2.1 Selecting Data from One Table

SQL DML statements



SELECT

INSERT

UPDATE

DELETE

Data **M**anipulation **L**anguage

Sometimes, `select` statements are not considered **DML**. **We are going to consider it DML.**

SQL DDL statements

CREATE DATABASE, TABLE, INDEX

ALTER TABLE, INDEX

DROP DATABASE, TABLE, INDEX

Data **D**efinition **L**anguage

This is used to create or remove objects from the database. Objects are tables, indexes, and such.

Columns, not fields.

books					
id	title	pagecount	genre	authorid	pubid
1	It	1138	Horror	10	100
2	Hamlet	500	Tragedy	13	103
3	I Know Why the Caged Bird Sings	304	Autobiogra phical	14	102
4	A Suitable Boy	1349	Drama/ Romance	15	103
5	The Joy Luck Club	288	Drama	16	104

A **primary key** uniquely
identifies a row in a table.

Rows, not records

```
-- drop schema if exists teachers CASCADE;
```

```
CrEaTe ScHeMa IF NOT EXISTS teachers;
```

```
-- Listing 2-2: Creating a table named teachers with six columns
```

```
CREATE TABLE if not exists teachers.teachers (  
    id bigserial don't use this type  
    id integer generated by default as identity primary key  
    , first_name varchar(25)  
    , last_name varchar(50)  
    , school varchar(50)  
    , hire_date date  
    , salary numeric  
);
```

```
-- This command will remove (drop) the table.
```

```
-- DROP TABLE teachers.teachers;
```

-- Listing 2-3 Inserting data into the teachers table

```
INSERT INTO teachers.teachers (  
    first_name, last_name, school, hire_date, salary)  
VALUES  
    ('Janet', 'Smith', 'F.D. Roosevelt HS', '2011-10-30', 36200)  
    , ('Lee', 'Reynolds', 'F.D. Roosevelt HS', '1993-05-22', 65000)  
    , ('Samuel', 'Cole', 'Myers Middle School', '2005-08-01', 43500)  
    , ('Samantha', 'Bush', 'Myers Middle School', '2011-10-30', 36200)  
    , ('Betty', 'Diaz', 'Myers Middle School', '2005-08-30', 43500)  
    , ('Kathleen', 'Roush', 'F.D. Roosevelt HS', '2010-10-22', 38500)  
    , ('Lester', 'Sapp', 'T.D. Roosevelt HS', NULL, 78500)  
;
```

```
SELECT * FROM teachers.teachers;
```

id	first_name	last_name	school	hire_date	salary
1	Janet	Smith	F.D. Roosevelt HS	2011-10-30	36,200
2	Lee	Reynolds	F.D. Roosevelt HS	1993-05-22	65,000
3	Samuel	Cole	Myers Middle School	2005-08-01	43,500
4	Samantha	Bush	Myers Middle School	2011-10-30	36,200
5	Betty	Diaz	Myers Middle School	2005-08-30	43,500
6	Kathleen	Roush	F.D. Roosevelt HS	2010-10-22	38,500

A **primary key** uniquely identifies a row in a table.

*-- Create the table with a **Natural Key***

```
CREATE TABLE if not exists teachers.teachers_nk (  
    first_name varchar(25)  
    , last_name varchar(50)  
    , school varchar(50)  
    , hire_date date  
    , salary numeric  
    , primary key(first_name, last_name)  
);
```

-- This command will remove (drop) the table.

-- DROP TABLE analysis.teachers2;

-- Listing 2-3 Inserting data into the teachers2 table

```
INSERT INTO teachers.teachers_nk (  
    first_name, last_name, school, hire_date, salary)  
VALUES  
    ('Janet', 'Smith', 'F.D. Roosevelt HS', '2011-10-30', 36200)  
    , ('Lee', 'Reynolds', 'F.D. Roosevelt HS', '1993-05-22', 65000)  
    , ('Samuel', 'Cole', 'Myers Middle School', '2005-08-01', 43500)  
    , ('Samantha', 'Bush', 'Myers Middle School', '2011-10-30', 36200)  
    , ('Betty', 'Diaz', 'Myers Middle School', '2005-08-30', 43500)  
    , ('Kathleen', 'Roush', 'F.D. Roosevelt HS', '2010-10-22', 38500)  
    , ('Lester', 'Sapp', 'T.D. Roosevelt HS', NULL, 78500)  
;
```


Using a **Natural Key** for the table

This is a PostgreSQL only syntax for selecting data from a table. I discourage it.

TABLE teachers.teachers_n

first_name	last_name	school	hire_date	salary
Janet	Smith	F.D. Roosevelt HS	2011-10-30	36,200
Lee	Reynolds	F.D. Roosevelt HS	1993-05-22	65,000
Samuel	Cole	Myers Middle School	2005-08-01	43,500
Samantha	Bush	Myers Middle School	2011-10-30	36,200
Betty	Diaz	Myers Middle School	2005-08-30	43,500
Kathleen		F.D. Roosevelt HS	2010-10-22	38,500

A **primary key** uniquely identifies a row in a table.

The basic syntax of the **SELECT** statement

```
SELECT select_list  
[FROM table_source]  
[WHERE search_condition]  
[ORDER BY order_by_list]  
[LIMIT {row_limit | ALL}]  
[OFFSET number]
```

The clauses **MUST** be given in this order.

The six clauses of the SELECT statement, for PostgreSQL

1. **SELECT**

2. **FROM**

3. **WHERE**

4. **ORDER BY**

5. **LIMIT**

6. **OFFEST**

} Non-ANSI

For now!

Most Basic **SELECT** Syntax

```
SELECT *  
FROM teachers . teachers ;
```

The **asterisk** following the **SELECT** keyword is a **wildcard**, which is like a stand-in for a value: it doesn't represent anything in particular and instead represents everything that value could possibly be. Here, it's shorthand for "select all columns."

It is not really a regex metacharacter.

Basic SELECT Syntax – More Detail

The particular DML we wish to execute, a **select** statement.

The **table_source**, the table from which we wish to select data. In this case, it is composed of both a schema name and a table name.



```
SELECT * FROM teachers.teachers;
```

Shorthand for all columns.
Later, this will turn into the **select_list** clause.

```
SELECT * FROM teachers.teachers ;
```

By the standards of SQL, when you use an asterisk in your `select_list`, **the order in which the columns are returned is not guaranteed**. The order of the columns is “**probably**” going to match the order in which they are defined in the table, but that is not guaranteed. It could even change.

If you really care about the order of the columns, you specify the column order in the `select_list`.

Querying a Subset of Columns

When you either do not need all the columns or you want the columns returned in a specific order, you list the column names in the **select_list**.

The **select_list** containing a list of columns you wish to return.

SELECT

last_name, first_name, salary
FROM teachers.teachers;

```
SELECT  
  last_name, first_name, salary  
FROM teachers.teachers;
```

last_name	first_name	salary
Smith	Janet	36,200
Reynolds	Lee	65,000
Cole	Samuel	43,500
Bush	Samantha	36,200
Diaz	Betty	43,500
Roush	Kathleen	38,500

Sorting Data with **ORDER BY**

By the standards of SQL, if an order to the returned data are not identified in the **select** statement, it **can be returned in any order**. The order of the data may change on subsequent select statements.

If you care about the order in which the data are returned, you need to specify an **ORDER BY** clause with an **order_by_list**.


```
SELECT  
    first_name, last_name, salary  
FROM teachers.teachers  
ORDER BY salary DESC;
```

By default, **ORDER BY** sorts values in ascending order (low to high), but here I sort in descending order by adding the **DESC** keyword (high to low). (The optional **ASC** keyword specifies sorting in ascending order.)

```
SELECT first_name, last_name, salary
FROM teachers.teachers
ORDER BY salary DESC;
```

first_name	last_name	salary
Lee	Reynolds	65,000
Samuel	Cole	43,500
Betty	Diaz	43,500
Kathleen	Roush	38,500
Janet	Smith	36,200
Samantha	Bush	36,200

```
SELECT
    first_name, last_name, salary
FROM teachers
```

ORDER BY salary;

Changing the sort by order. It could also have **ASC**.

```
SELECT first_name, last_name, salary
FROM teachers.teachers
ORDER BY salary ;
```

first_name	last_name	salary
Janet	Smith	36,200
Samantha	Bush	36,200
Kathleen	Roush	38,500
Samuel	Cole	43,500
Betty	Diaz	43,500
Lee	Reynolds	65,000

Sorting Data by More than Column

```
SELECT  
    last_name, school, hire_date  
FROM teachers.teachers  
ORDER BY  
    school ASC  
    , hire_date DESC;
```

Notice that not all columns need be sorted in the same order. Here, the initial column is ascending and the following one is descending order.

```
SELECT last_name, school, hire_date
FROM teachers.teachers
ORDER BY school ASC, hire_date DESC;
```

last_name	school	hire_date
Smith	F.D. Roosevelt HS	2011-10-30
Roush	F.D. Roosevelt HS	2010-10-22
Reynolds	F.D. Roosevelt HS	1993-05-22
Bush	Myers Middle School	2011-10-30
Diaz	Myers Middle School	2005-08-30
Cole	Myers Middle School	2005-08-01

Using **DISTINCT** to Find Unique Values

It is not unusual for a column to contain rows with duplicate values. In the `teachers` table, the `school` column lists the same school names multiple times because each school employs many teachers.

We can use the **DISTINCT** keyword as part of a query that eliminates duplicates and **shows only unique values**.

```
SELECT DISTINCT school  
FROM teachers.teachers  
ORDER BY school;
```

```
SELECT DISTINCT school  
FROM teachers.teachers  
ORDER BY school;
```

school
F.D. Roosevelt HS
Myers Middle School

Using **DISTINCT** to Find Unique Values

It is important to be aware that the **DISTINCT** keyword **applies to the entire row returned, not to an individual column.**

```
SELECT  
    DISTINCT school, salary  
FROM teachers.teachers  
ORDER BY school, salary;
```



```
SELECT
  DISTINCT school, salary
FROM teachers.teachers
ORDER BY school, salary;
```

Each **DISTINCT** school and salary combination.

school	salary
F.D. Roosevelt HS	36,200
F.D. Roosevelt HS	38,500
F.D. Roosevelt HS	65,000
Myers Middle School	36,200
Myers Middle School	43,500

Filtering Rows with **WHERE**

You do not always need to return **all** rows of data from a table.

Sometimes, you'll want to **limit the rows a query returns to only those in which one or more columns meet certain criteria.**

You want to apply a filter on the data.

```
SELECT  
    last_name, school, hire_date  
FROM teachers.teachers  
WHERE  
    school = 'Myers Middle School';
```

Return only those rows that have “Meyers Middle School” as the school name.

```
SELECT  
    last_name, school, hire_date  
FROM teachers  
WHERE  
    school = 'Myers Middle School';
```



NOTE: Strings used in **WHERE** clauses MUST make use of **single quote characters** **NOT** double quote characters.

```
SELECT last_name, school, hire_date  
FROM teachers.teachers  
WHERE school = 'Myers Middle School'
```

Returns only the rows which have 'Myers Middle School' in the **school** column.

last_name	school	hire_date
Cole	Myers Middle School	2005-08-01
Bush	Myers Middle School	2011-10-30
Diaz	Myers Middle School	2005-08-30

```
SELECT  
    last_name, school, hire_date  
FROM teachers
```

WHERE

```
    school = "Myers Middle School";
```



**SQL Error [42703]: ERROR: column "Myers Middle School"
does not exist**

Position: 75

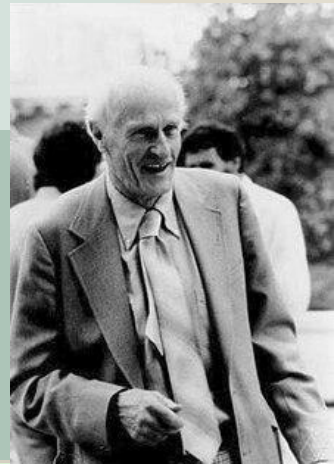
Error position: line: 60 pos: 74

Operator	Function	Example
=	Equal to	WHERE school = 'Baker Middle'
<> Or !=	Not equal to	WHERE school <> 'Baker Middle'
>	Greater than	WHERE salary > 20000
<	Less than	WHERE salary < 20000
>=	Greater that or equal	WHERE salary >= 20000
<=	Less than or equal	WHERE salary <= 20000
between	Within a range (inclusive)	WHERE salary BETWEEN 20000 AND 40000
in	Match one of a set of values	WHERE last_name IN ('Bush', 'Roush') In is case sensitive
like	Match a pattern (case sensitive)	WHERE first_name LIKE 'Sam%'
ilike	Match a pattern (case insensitive)	WHERE first_name ILIKE 'sam%'
not	Negates a condition	WHERE first_name NOT ILIKE 'sam%'

So we can say
<> == !=

Using **LIKE** and **ILIKE** with **WHERE**

The regular expression meta-characters used in SQL are not exactly like the meta-characters you may be accustomed in more common regular expressions, from Stephen Kleene.



- **Percent sign (%)** A wildcard matching **zero or more characters**
 - **Underscore (_)** A wildcard matching **just one character**
- `LIKE 'b%'` `-- a 'b' followed by one or more chars`
 - `LIKE '%ak%'` `-- match anything with a 'ak' somewhere`
 - `LIKE '_aker'` `-- a single character before 'aker'`
 - `LIKE 'ba_er'` `-- match any single character between ba and er`

More **LIKE**

```
'abc' LIKE 'abc' -- true
'abc' LIKE 'a%' -- true
'abc' LIKE '_b_' -- true
'abc' LIKE 'c' -- false

'ABC' LIKE 'abc' -- false in PostgreSQL
```

By default, **PostgreSQL** performs a **case-sensitive** string comparison.

MySQL and **MS SQL Server** have **case-insensitive** behaviors by default.

PostgreSQL and **LIKE** vs **ILIKE**

The PostgreSQL keyword **ILIKE** can be used instead of **LIKE** to make the match case insensitive according to the active locale.

This is not in the SQL standard but is a PostgreSQL extension.

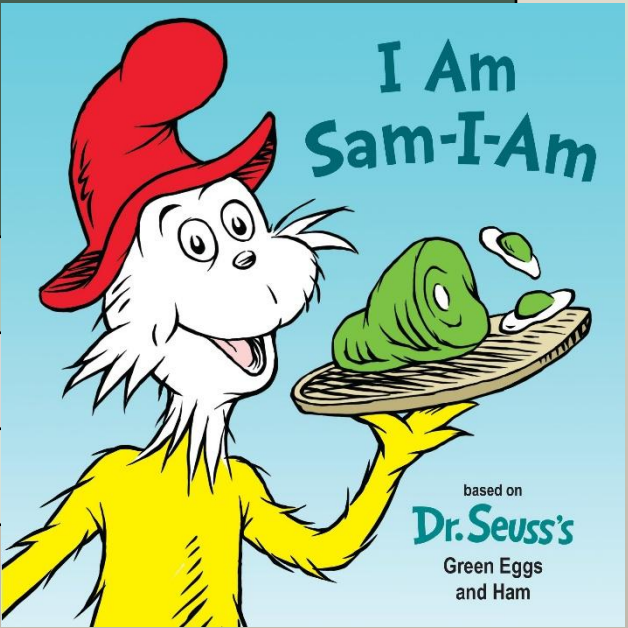
```
select *  
from teachers.teachers  
where first_name NOT ILIKE 'Jesse';
```

```
SELECT first_name, last_name
FROM teachers.teachers
WHERE first_name LIKE 'sam%';
```

first_name	last_name
------------	-----------

```
SELECT first_name, last_name
FROM teachers.teachers
WHERE first_name ILIKE 'sam%';
```

first_name	last_name
Samuel	Cole
Samantha	Bush



Combining Operators with **AND** and **OR** in the WHERE Clause

```
SELECT *  
FROM teachers.teachers  
WHERE school = 'Myers Middle School'  
      AND salary < 40000;
```

```
SELECT *  
FROM teachers.teachers  
WHERE last_name = 'Cole'  
      OR last_name = 'Bush';
```

```
SELECT *  
FROM teachers.teachers  
WHERE school = 'F.D. Roosevelt HS'  
      AND (salary < 38000 OR salary > 40000);
```

```
SELECT last_name, school, hire_date
FROM teachers.teachers
WHERE school = 'Myers Middle School'
and hire_date < '2010-01-01';
```

Returns only the rows from 'Myers Middle School' with teachers hired before January 1st 2010.

last_name	school	hire_date
Cole	Myers Middle School	2005-08-01
Diaz	Myers Middle School	2005-08-30

```
SELECT distinct school  
FROM teachers.teachers  
WHERE school <> 'F.D. Roosevelt'
```

Select the unique school names that are
not “F.D. Roosevelt”

school

Myers Middle School

T.D. Roosevelt HS

The **LIMIT** Clause

Sometimes, you **don't need to return all the rows** of data, even when you filter data using a **WHERE** clause.

Using the **LIMIT** clause, you can return no more than the number of rows specified by the **LIMIT** value.

```
SELECT last_name, school, hire_date  
FROM teachers.teachers  
LIMIT 2;
```

Non-ANSI

```
SELECT last_name, school, hire_date
FROM teachers.teachers
LIMIT 2;
```

last_name	school	hire_date
Smith	F.D. Roosevelt HS	2011-10-30
Reynolds	F.D. Roosevelt HS	1993-05-22

The **OFFSET** Clause

Sometimes, you want to jump ahead into the data returned from a **SELECT** statement. You can do this with the **OFFSET** clause.

```
SELECT last_name, school, hire_date  
FROM teachers.teachers  
OFFSET 2;
```

Non-ANSI

The **OFFSET** Clause

```
SELECT last_name, school, hire_date  
FROM teachers.teachers OFFSET 2;
```

last_name	school	hire_date
Cole	Myers Middle School	2005-08-01
Bush	Myers Middle School	2011-10-30
Diaz	Myers Middle School	2005-08-30
Roush	F.D. Roosevelt HS	2010-10-22

```
SELECT last_name, school, hire_date  
FROM teachers.teachers;
```

last_name	school	hire_date
Smith	F.D. Roosevelt HS	2011-10-30
Reynolds	F.D. Roosevelt HS	1993-05-22
Cole	Myers Middle School	2005-08-01
Bush	Myers Middle School	2011-10-30
Diaz	Myers Middle School	2005-08-30
Roush	F.D. Roosevelt HS	2010-10-22

Combining **LIMIT** and **OFFSET**

```
SELECT last_name, school, hire_date  
FROM teachers.teachers LIMIT 2 OFFSET 1;
```

last_name	school	hire_date
Reynolds	F.D. Roosevelt HS	1993-05-22
Cole	Myers Middle School	2005-08-01

```
SELECT last_name, school, hire_date  
FROM teachers.teachers;
```

last_name	school	hire_date
Smith	F.D. Roosevelt HS	2011-10-30
Reynolds	F.D. Roosevelt HS	1993-05-22
Cole	Myers Middle School	2005-08-01
Bush	Myers Middle School	2011-10-30
Diaz	Myers Middle School	2005-08-30
Roush	F.D. Roosevelt HS	2010-10-22
Sapp	T.D. Roosevelt HS	

Some Other Things We can do with **SELECT**

Sometimes, you want to combine the data from multiple text based columns into a single column in the result set. You can do this with the ANSI concatenation operator **||** (double vertical bars, pipes).

A common place to do this is to combine first and last names into a single column in the result set.

Some Other Things We can do with **SELECT**

SELECT The ANSI standard **concatenate** operator.

```
first_name || ' ' || last_name, salary, salary + 1000
FROM teachers.teachers;
```

?column?	salary	?column?
Janet Smith	36,200	37,200
Lee Reynolds	65,000	66,000
Samuel Cole		
Samantha Bush	36,200	37,200
Betty Diaz	43,500	44,500
Kathleen Roush	38,500	39,500

In PostgreSQL, columns that are the result of an expression are named **?column?** in the result set.

Some Other Things We can do with **SELECT**

The columns named **?column?** in your result set is just ugly. You can improve the appearance of your result set by renaming the columns with expressions, or any other column.

You use the **AS** syntax to create an alias for the column:

```
SELECT
```

```
    first_name || last_name AS "Teacher Name"
```

```
    , salary AS "Old Salary"
```

```
    , salary + 1000 AS "New Salary"
```

```
FROM teachers.teachers;
```

Note the double quotes
in the alias column name.

Some Other Things We can do with **SELECT**

SELECT

first_name || last_name as "Teacher Name"

, salary as "Old Salary"

, salary + 1000 as "New Salary"

FROM teachers.teachers

LIMIT 2;

Rename the columns in the result set.

Note the double quotes.

Teacher Name	Old Salary	New Salary
JanetSmith	36,200	37,200
LeeReynolds	65,000	66,000

What is this **NULL** Thingie?

In SQL, **null** or **NULL** is a special marker used to indicate that a data value does not exist in the database. Introduced by the creator of the relational database model, E. F. Codd, SQL **NULL** serves to fulfill the requirement that all true relational database management systems (RDBMS) support a representation of "missing information and inapplicable information".

In SQL, **NULL** is a marker, not a value. This usage is quite different from most programming languages, where a **null** value of a reference means it is not pointing to any object.

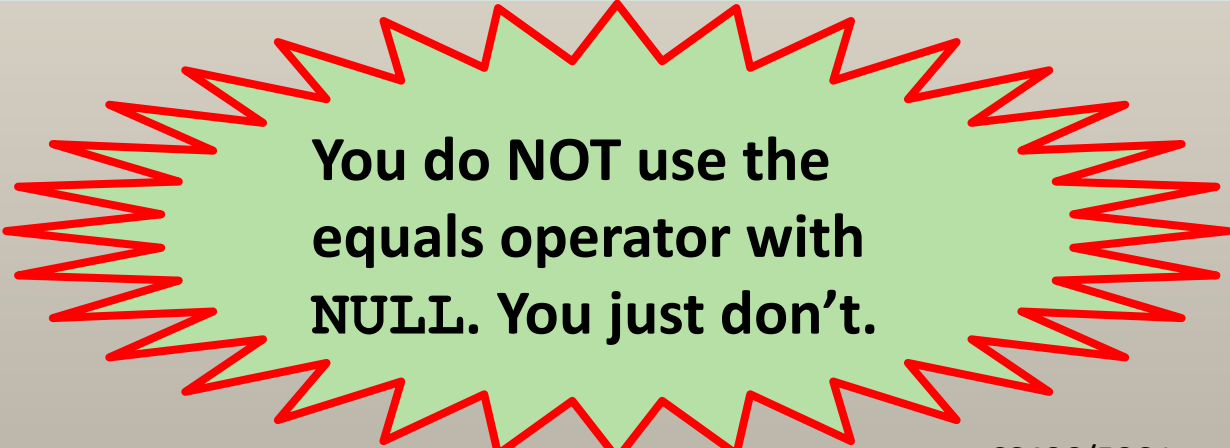
[https://en.wikipedia.org/wiki/Null_\(SQL\)](https://en.wikipedia.org/wiki/Null_(SQL))

Using of NULL in SELECT

The syntax of the **WHERE** clause with the **IS NULL** or **IS NOT NULL** comparison is *different*...

When comparing to **NULL**, you **do not use the = or <> operators.**

You use the **IS NULL** or **IS NOT NULL** comparisons.



You do NOT use the equals operator with NULL. You just don't.

A **NULL** in the where Clause

Using **NULL** in the **where** clause

```
select *  
from teachers.teachers  
where hire_date is null;
```

```
select *  
from teachers.teachers  
where hire_date is not null;
```

```
select *  
from teachers.teachers where hire_date is null;
```

id	first_name	last_name	school	hire_date	salary
7	Lester	Sapp	T.D. Roosevelt HS		78,500

```
select *  
from teachers.teachers where hire_date is not null;
```

id	first_name	last_name	school	hire_date	salary
1	Janet	Smith	F.D. Roosevelt HS	2011-10-30	36,200
2	Lee	Reynolds	F.D. Roosevelt HS	1993-05-22	65,000
3	Samuel	Cole	Myers Middle School	2005-08-01	43,500
4	Samantha	Bush	Myers Middle School	2011-10-30	36,200
5	Betty	Diaz	Myers Middle School	2005-08-30	43,500
6	Kathleen	Roush	F.D. Roosevelt HS	2010-10-22	38,500